# E-commerce Data Analytics: TeddyWorld Case Study

This notebook utilizes transaction data from March 2012 to March 2015 from the TeddyWorld store to develop a strategic report. It focuses on the objectives of maximizing the customer experience and evaluating A/B testing results to propose revenue growth strategies.

Datasets Used in the Analysis:

- **orders.csv:** Stores high-level, general information regarding placed orders.
- **order_items.csv:** Contains line-item details for specific products within each order.
- **order_item_refunds.csv:** Tracks refund transactions processed for customers.
- **products.csv:** The catalog of products available on the platform.
- **website_sessions.csv:** Logs data regarding user visits and browsing sessions on the website.
- **website_pageviews.csv:** Records granular details of the individual pages viewed by users during a session.

## I. Set Up & Data Ingestion

```
from google.colab import drive
drive.mount('/content/drive')
```

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

```
# Loading data
data_path = '/content/drive/MyDrive/Analytics Report/Data/'

orders = pd.read_csv(data_path + 'orders.csv')
sessions = pd.read_csv(data_path + 'website_sessions.csv')
products = pd.read_csv(data_path + 'products.csv')
order_items = pd.read_csv(data_path + 'order_items.csv')
refunds = pd.read_csv(data_path + 'order_item_refunds.csv')
pageviews = pd.read_csv(data_path + 'website_pageviews.csv')
```

```
print("Data loaded successfully!")
```

## II. Data Cleaning & Pre-processing

```
# Convert datetime format and generate "month" column for dataframes
for df in [orders, sessions, products, order_items, refunds, pageviews]:
    df['created_at'] = pd.to_datetime(df['created_at'])
    df['month'] = df['created_at'].dt.to_period('M').astype(str)

# Handling nulls
sessions['utm_source'] = sessions['utm_source'].fillna('organic')
sessions['utm_campaign'] = sessions['utm_campaign'].fillna('none')
sessions['utm_content'] = sessions['utm_content'].fillna('none')
sessions['http_referer'] = sessions['http_referer'].fillna('direct_traffic')

print('Date processed!')
```

## III. General Overview Metrics

```
# Calculate key metrics
total_sessions = sessions['website_session_id'].nunique()
total_orders = orders['order_id'].nunique()
total_revenue = orders['price_usd'].sum()
total_refunds = refunds['refund_amount_usd'].sum()
total_net_revenue = total_revenue - total_refunds
total_cogs = orders['cogs_usd'].sum()
total_users = orders['user_id'].nunique()

print('Calculated successfully!')
```

```
overall_cr = (total_orders / total_sessions) * 100
overall_gross_profit = total_net_revenue - total_cogs
overall_gross_margin = (overall_gross_profit / total_net_revenue) * 100
avg_order_value = total_revenue / total_orders
arpu = total_net_revenue / total_users

print('Calculated successfully!')
```

```
summary_metrics = pd.DataFrame({
    'Metric': [
        'Sessions', 'Orders', 'Revenue',
        'Refund Amount', 'Net Revenue', 'Conversion Rate',
        'Gross Profit', 'Gross Profit Margin', 'AOV', 'ARPU'
    ],
    'Value': [
        f"{total_sessions:,}", f"{total_orders:,}",
        f"${total_revenue:,.2f}", f"${total_refunds:,.2f}",
        f"${total_net_revenue:,.2f}", f"{overall_cr:.2f}%",
        f"${overall_gross_profit:,.2f}", f"{overall_gross_margin:.2f}%",
        f"${avg_order_value:,.2f}", f"${arpu:,.2f}"
    ]
})

print('--- OVERVIEW OF BUSINESS METRICS ---')
display(summary_metrics)
```

## IV. Conversion Rate Analysis

```
# INDEX DECOMPOSITION USING A LOGIC TREE

# 1.1 List of pages belonging to each step of the funnel.
product_pages = [
    '/the-original-mr-fuzzy',
    '/the-forever-love-bear',
    '/the-birthday-sugar-panda',
    '/the-hudson-river-mini-bear'
]
checkout_pages = ['/shipping', '/billing', '/billing-2']

# 1.2 Flag the steps based on the URL.
pageviews['is_product_view'] = pageviews['pageview_url'].isin(product_pages)
pageviews['is_add_to_cart'] = pageviews['pageview_url'] == '/cart'
pageviews['is_checkout'] = pageviews['pageview_url'].isin(checkout_pages)

# 1.3 Group the sessions by website_session_id
funnel_steps = pageviews.groupby('website_session_id').agg(
    viewed_product=('is_product_view', 'max'),
    added_to_cart=('is_add_to_cart', 'max'),
    initiated_checkout=('is_checkout', 'max')
).reset_index()
```

```
# 1.4 Verify a successful purchase by checking the session_id in the orders table
purchased_sessions = orders['website_session_id'].unique()
funnel_steps['purchased'] = funnel_steps['website_session_id'].isin(purchased_sessions)
```

```
# SEPARATE ACCORDING TO CUSTOMER TYPE

# Get the flag from the website_sessions
session_info = sessions[['website_session_id', 'is_repeat_session']]

# Merge funnel data with session information.
df_master = pd.merge(session_info, funnel_steps, on='website_session_id', how='left')

# Web sessions but no pageviews (fill False)
cols_to_fill = ['viewed_product', 'added_to_cart', 'initiated_checkout', 'purchased']
df_master[cols_to_fill] = df_master[cols_to_fill].fillna(False)

# New Customer / Returning Customer
df_master['customer_type'] = np.where(df_master['is_repeat_session'] == 1, 'Returning Customer', 'New Customer')
```

```
# METRIC TREE CALCULATED FUNCTION
def calc_metric_tree(df):
    landed = len(df)
    views = df['viewed_product'].sum()
    carts = df['added_to_cart'].sum()
    checkouts = df['initiated_checkout'].sum()
    orders = df['purchased'].sum()

    return pd.Series({
        '0. Landed': landed,
        '1. Product Views': views,
        '2. Add to Carts': carts,
        '3. Checkouts': checkouts,
        '4. Purchases (Orders)': orders,

        'Root: Overall CR (%)': round((orders / landed * 100) if landed > 0 else 0, 2),

        'Branch 1: View Rate (%)': round((views / landed * 100) if landed > 0 else 0, 2),
        'Branch 2: Add-to-cart Rate (%)': round((carts / views * 100) if views > 0 else 0, 2),
        'Branch 3: Checkout Rate (%)': round((checkouts / carts * 100) if carts > 0 else 0, 2),
        'Branch 4: Purchase Rate (%)': round((orders / checkouts * 100) if checkouts > 0 else 0, 2)
    })
```

```
# EXPORT REPORT
```

```
print("="*60)
print(" ROOT & BRANCHES (ALL)")
print("="*60)
overall_tree = calc_metric_tree(df_master).to_frame(name='All')
display(overall_tree)

print("\n" + "="*60)
print(" LEAVES (BY CUSTOMER TYPE)")
print("="*60)
leaves_tree = df_master.groupby('customer_type').apply(calc_metric_tree).T
display(leaves_tree)
```

> **Branch 1:** Only 44.45% of total traffic reached the product detail page. This indicates that over 55% of visitors dropped off (bounced) directly from the homepage, category pages, or article pages. The discrepancy between the segments in this branch is highly notable: New customers (43.22%) have a significantly lower rate of navigating to product pages compared to returning customers (50.68%). Because the new customer segment accounts for a massive 83% of the total traffic (394,318 sessions), their downward pull on the overall conversion rate is exceptionally strong.

> **Branch 4:** Only 50.11% of customers who initiated the checkout process successfully completed their orders. Losing half of the users at this stage of "highest purchase intent" represents a massive loss in potential revenue. Notably, both new customers (50.02%) and returning customers (50.51%) drop off at a remarkably similar rate. This indicates that the root cause does not lie in brand trust or user habits, but rather stems directly from friction within the checkout experience itself.

## Analysis by Device Type

```
session_info_device = sessions[['website_session_id', 'device_type']]
df_master_device = pd.merge(session_info_device, funnel_steps, on='website_session_id', how='left')

cols_to_fill = ['viewed_product', 'added_to_cart', 'initiated_checkout', 'purchased']
df_master_device[cols_to_fill] = df_master_device[cols_to_fill].fillna(False)

print("="*55)
print(" LEAVES (DEVICE TYPE)")
print("="*55)
leaves_tree_device = df_master_device.groupby('device_type').apply(calc_metric_tree).T
display(leaves_tree_device)
```

```
# VISUALIZE "LEAVES (DEVICE TYPE)" TABLE

# 1. Extract a list of devices from the data table
devices = leaves_tree_device.columns.tolist()
device_1 = devices[0]
device_2 = devices[1]

# 2. Declare the 5 steps of the funnel
stages = [
    '0. Landed',
    '1. Product Views',
    '2. Add to Carts',
    '3. Checkouts',
    '4. Purchases (Orders)'
]

# 3. Create a dedicated 1-row, 2-column Subplot layout for Funnels
fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=(f"{str(device_1).upper()}", f"{str(device_2).upper()}"),
    specs=[[{"type": "funnel"}, {"type": "funnel"}]]
)

# 4. Funnel for first device (Column 1)
fig.add_trace(go.Funnel(
    name = str(device_1).title(),
    y = stages,
    x = leaves_tree_device[device_1][stages].tolist(),
    textinfo = "value+percent initial+percent previous",
    marker = {"color": "#6CB4EE"} # light blue
), row=1, col=1)

# 5. Funnel for second device (Column 2)
fig.add_trace(go.Funnel(
    name = str(device_2).title(),
    y = stages,
    x = leaves_tree_device[device_2][stages].tolist(),
    textinfo = "value+percent initial+percent previous",
    marker = {"color": "#F4B28F"} # light orange
), row=1, col=2)

# 6. Configure the overall title and interface
fig.update_layout(
    title_text='<b>Comparison of Convension Funnels: Desktop vs Mobile</b><br><sup>Visualize CR on each device</sup>',
    height=600,
    showlegend=False,
```

```
    font_size=12
)

fig.update_yaxes(showticklabels=False, row=1, col=2)

fig.show()
```

> **INSIGHT**: Comparing the two charts, it's clear that the mobile funnel is significantly narrower, confirming that the mobile shopping experience is problematic.

## Analysis by Landing page on Mobile device

```
# ======================================================================
# STEP 1: FIRST LANDING PAGE OF EACH SESSION
# ======================================================================
# Sorting pageview
landing_pages = pageviews.sort_values('website_pageview_id').drop_duplicates(subset='website_session_id', keep='first')

# Rename columns and remove columns
landing_pages = landing_pages[['website_session_id', 'pageview_url']].rename(columns={'pageview_url': 'landing_page'})
```

```
# ======================================================================
# STEP 2: MERGE & FILTER
# ======================================================================
# Merging to df_master_device
df_master_lp = pd.merge(df_master_device, landing_pages, on='website_session_id', how='left')

# Filtering mobile device only
df_mobile_lp = df_master_lp[df_master_lp['device_type'] == 'mobile']
```

```
# ======================================================================
# STEP 3: EXPORT REPORT
# ======================================================================
print("="*65)
print("LANDING PAGE (MOBILE Only)")
print("="*65)

# calc_metric_tree
leaves_tree_lp = df_mobile_lp.groupby('landing_page').apply(calc_metric_tree).T
display(leaves_tree_lp)
```

```
# ======================================================================
# VISUALIZING ANALYSIS RESULTS
# ======================================================================
# Transpose
df_plot = leaves_tree_lp.T

# Descending Landing Page by Traffic
df_plot = df_plot.sort_values(by='0. Landed', ascending=False)

# Extract data axes
x_landing_pages = df_plot.index.tolist()
y_traffic = df_plot['0. Landed'].tolist()
y_view_rate = df_plot['Branch 1: View Rate (%)'].tolist()
```

```
# DUAL Y-AXIS

fig = make_subplots(specs=[[{"secondary_y": True}]])

# Left y-axis
fig.add_trace(
    go.Bar(
        x = x_landing_pages,
        y = y_traffic,
        name = "Traffic",
        marker_color = '#6CB4EE', # light blue
        text = [f"{v:,.0f}" for v in y_traffic], # Display the number at the top of the column
        textposition = 'auto'
    ),
    secondary_y = False, # Attach to the y-axis (left side)
)

# Right y-axis
fig.add_trace(
    go.Scatter(
        x = x_landing_pages,
        y = y_view_rate,
        name = "View Rate",
        mode = 'lines+markers+text', # present lines, dots and %
        marker = dict(color='#E55A44', size=8), # striking orange-red
        line = dict(width=3),
        text = [f"{v:.1f}%" for v in y_view_rate], # present %
        textposition = 'top center' # move the text
    ),
    secondary_y = True, # Attach to the y-axis (right side)
```

```
)

# Configure layout
fig.update_layout(
    title_text = "<b>Efective landing page on mobile</b><br><sup>Website traffic and user product view rate</sup>",
    font_size = 12,
    height = 600,
    hovermode = "x unified", # hover for information
    legend = dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1) # legend position
)

# Configure headers for the y-axes.
fig.update_yaxes(title_text="Sessions", secondary_y=False, showgrid=False)
fig.update_yaxes(title_text="View Rate", secondary_y=True, range=[0, max(y_view_rate) * 1.2], showgrid=False)

fig.show()
```

> **INSIGHT:** Lander 3 is currently the best-performing landing page, yet its highest View Rate peaks at merely 34.5%. This implies that even in the best-case scenario, TeddyWorld is still losing over 65% of potential customers right at the entry point. When compared to the 50% rate on Desktop (from previous analyses), the browsing and product discovery experience across the entire mobile platform requires a thorough audit and re-evaluation.

## Bottom funnel analysis

```
# ======================================================================
# MICRO-FUNNEL ANALYSIS FUNCTION
# ======================================================================
def analyze_checkout_dropoff(device_type):
    # 1. Filter Session ID by device type
    sessions_by_device = sessions[sessions['device_type'] == device_type]['website_session_id']

    # 2. Filter Pageviews
    checkout_urls = ['/shipping', '/billing', '/billing-2']
    checkout_pv = pageviews[(pageviews['website_session_id'].isin(sessions_by_device)) &
                            (pageviews['pageview_url'].isin(checkout_urls))]

    # 3. List of successful purchasing Sessions
    purchased_sessions = orders['website_session_id'].unique()

    # 4. Calculate for each URL
    results = []
```

```
    for url in checkout_urls:
        sessions_at_url = checkout_pv[checkout_pv['pageview_url'] == url]['website_session_id'].unique()
        vol = len(sessions_at_url)

        # Rename variable to avoid error
        num_orders = np.isin(sessions_at_url, purchased_sessions).sum()

        if vol > 0:
            results.append({
                'Trang Checkout': url,
                '1. Traffic (Volume)': vol,
                '2. Purchased Orders': num_orders,  # update new variable
                '3. Drop-offs': vol - num_orders,  # update new variable
                '4. Purchase Rate': round((num_orders / vol * 100), 2) # update new variable
            })

    # 5. Format to DataFrame
    df_result = pd.DataFrame(results).set_index('Trang Checkout')
    return df_result
```

```
# ======================================================================
# CALL FUNCTIONS AND OUTPUT RESULTS
# ======================================================================
print("="*65)
print(" BOTTOM FUNNEL RESULT (CHECKOUT PAGE) - DEVICE: DESKTOP")
print("="*65)
display(analyze_checkout_dropoff('desktop'))

print("\n" + "="*65)
print(" BOTTOM FUNNEL RESULT (CHECKOUT PAGE) - DEVICE: MOBILE")
print("="*65)
display(analyze_checkout_dropoff('mobile'))
```

```
# ======================================================================
# PURCHASE RATE COMPARISON
# ======================================================================

df_desktop_chk = analyze_checkout_dropoff('desktop')
df_mobile_chk = analyze_checkout_dropoff('mobile')
```

```
# VISUALIZING THE BOTTOM FUNNEL OF TWO DEVICES

# List of Checkout pages (x-axis)
checkout_urls = df_desktop_chk.index.tolist()
```

```
fig = go.Figure()

fig.add_trace(go.Bar(
    name='Desktop',
    x=checkout_urls,
    y=df_desktop_chk['4. Purchase Rate'],
    marker_color='#75D187', # green
    text=[f"{val}%" for val in df_desktop_chk['4. Purchase Rate']],
    textposition='auto',

    # Add hidden Volume and Drop-off data to display when hover
    customdata=df_desktop_chk[['1. Traffic (Volume)', '3. Drop-offs']],
    hovertemplate=(
        "<b>%{x} (Desktop)</b><br>"
        "Purchase Rate: %{y}%<br>"
        "Total traffic: %{customdata[0]:,.0f}<br>"
        "Drop-offs: %{customdata[1]:,.0f}<extra></extra>"
    )
))

# Mobile column
fig.add_trace(go.Bar(
    name='Mobile',
    x=checkout_urls,
    y=df_mobile_chk['4. Purchase Rate'],
    marker_color='#F4B28F', # light orange
    text=[f"{val}%" for val in df_mobile_chk['4. Purchase Rate']],
    textposition='auto',

    customdata=df_mobile_chk[['1. Traffic (Volume)', '3. Drop-offs']],
    hovertemplate=(
        "<b>%{x} (Mobile)</b><br>"
        "Purchase Rate: %{y}%<br>"
        "Total traffic: %{customdata[0]:,.0f}<br>"
        "Drop-offs: %{customdata[1]:,.0f}<extra></extra>"
    )
))

# Configure display
fig.update_layout(
    title_text="<b>Purchase Rates Comparison</b><br><sup>A/B Testing (/billing vs /billing-2) Evaluation between devices</sup>",
    barmode='group', # group column
    yaxis_title="Purchase rate (%)",
    font_size=12,
    height=550,
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
```

```
)

fig.update_yaxes(showgrid=False, rangemode="tozero")

fig.show()
```

> **INSIGHT:** The second checkout page variant demonstrates a significantly higher conversion rate compared to the original checkout page across both devices.

> **Action Plan:** Immediately roll out this new checkout page to 100% of the traffic.

```
# ======================================================================
# ASSESS ORDER QUALITY AT THE BOTTOM OF THE FUNNEL
# ======================================================================
# Filter pageviews belong to billing
billing_pages = ['/billing', '/billing-2']
billing_pv = pageviews[pageviews['pageview_url'].isin(billing_pages)]

# Retain the last URL billing record for each session (in case the user refreshes the page using F5)
billing_sessions = billing_pv.drop_duplicates(subset='website_session_id', keep='last')[['website_session_id', 'pageview_url']]
billing_sessions.rename(columns={'pageview_url': 'billing_version'}, inplace=True)
```

```
# Inner join orders with billing version to only retrieve purchase sessions
orders_with_billing = pd.merge(orders, billing_sessions, on='website_session_id', how='inner')

# Inner join with sessions for device information
orders_full_info = pd.merge(orders_with_billing, sessions[['website_session_id', 'device_type']], on='website_session_id', how='inner

# Adding column Margin = Revenue - COGS
orders_full_info['margin_usd'] = orders_full_info['price_usd'] - orders_full_info['cogs_usd']
```

```
# Group by device và billing version
aov_analysis = orders_full_info.groupby(['device_type', 'billing_version']).agg(
    total_orders=('order_id', 'count'),                    # Total orders
    total_revenue=('price_usd', 'sum'),                    # Total revenue
    aov_usd=('price_usd', 'mean'),                         # AOV
    avg_items_per_order=('items_purchased', 'mean'),       # Average number of products per order
    avg_margin_usd=('margin_usd', 'mean')                  # Average profit per order
).reset_index()
```

```
# Formatting
aov_analysis['total_revenue'] = aov_analysis['total_revenue'].apply(lambda x: f"${x:,.2f}")
aov_analysis['aov_usd'] = aov_analysis['aov_usd'].apply(lambda x: f"${x:,.2f}")
aov_analysis['avg_items_per_order'] = aov_analysis['avg_items_per_order'].round(2)
aov_analysis['avg_margin_usd'] = aov_analysis['avg_margin_usd'].apply(lambda x: f"${x:,.2f}")
```

```
# Rename column
aov_analysis.columns = [
    'Device', 'Billing Version', 'Purchased Orders',
    'Total revenue', 'AOV', 'Avg no. products per order', 'Avg profit per order'
]

print("="*85)
print(" ORDER QUALITY ANALYSIS TABLE (AOV) AT THE BOTTOM OF THE FUNNEL (A/B TESTING)")
print("="*85)
display(aov_analysis)
```

```
# VISUALIZE AOV COMPARISION RESULT

chart_data = orders_full_info.groupby(['device_type', 'billing_version']).agg(
    aov_usd=('price_usd', 'mean'),
    margin_usd=('margin_usd', 'mean'),
    total_orders=('order_id', 'count')
).reset_index()

# Creating two groups of columns.
df_old_billing = chart_data[chart_data['billing_version'] == '/billing'].set_index('device_type')
df_new_billing = chart_data[chart_data['billing_version'] == '/billing-2'].set_index('device_type')

# List of devices (x-axis)
devices = ['desktop', 'mobile']

# /BILLING page
fig = go.Figure()

fig.add_trace(go.Bar(
    name='/billing',
    x=[d.title() for d in devices], # Capitalize the first letter
    y=df_old_billing.loc[devices, 'aov_usd'],
    marker_color='#BDBDBD', # gray
    text=[f"${val:,.1f}" for val in df_old_billing.loc[devices, 'aov_usd']],
    textposition='auto',

    # Add Profit and Number of Orders
```

```
        customdata=df_old_billing.loc[devices, ['margin_usd', 'total_orders']],
        hovertemplate=(
            "<b>%{x} - /billing</b><br>"
            "AOV: $%{y:,.2f}<br>"
            "Avg profit per order: $%{customdata[0]:,.2f}<br>"
            "Total orders: %{customdata[1]}<extra></extra>"
        )
    ))

    # /BILLING-2 page
    fig.add_trace(go.Bar(
        name='/billing-2',
        x=[d.title() for d in devices],
        y=df_new_billing.loc[devices, 'aov_usd'],
        marker_color='#4A90E2', # striking blue
        text=[f"${val:,.1f}" for val in df_new_billing.loc[devices, 'aov_usd']],
        textposition='auto',

        customdata=df_new_billing.loc[devices, ['margin_usd', 'total_orders']],
        hovertemplate=(
            "<b>%{x} - /billing-2</b><br>"
            "AOV: $%{y:,.2f}<br>"
            "Avg profit per order: $%{customdata[0]:,.2f}<br>"
            "Total orders: %{customdata[1]}<extra></extra>"
        )
    ))

    # Configure display
    fig.update_layout(
        title_text="<b>AOV Assessment at the bottom of the funnelL</b><br><sup>Comparing the Average Order Value between 2 versions of the
        barmode='group',
        yaxis_title="AOV (USD)",
        font_size=12,
        height=550,
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )

    fig.update_yaxes(showgrid=False, rangemode="tozero")

    fig.show()
```

> **INSIGHT:** The /billing-2 page drives cross-selling and higher order value better than the /billing page.

## Revenue forecast

```
# ======================================================================
# DETERMINING A/B TESTING TIME
# ======================================================================
pageviews['created_at'] = pd.to_datetime(pageviews['created_at'])
ab_test_start = pageviews[pageviews['pageview_url'] == '/billing-2']['created_at'].min()
ab_test_end = pageviews['created_at'].max()

# Calculate the number of days (Use the max function to avoid errors if the data is not clean)
test_duration_days = max((ab_test_end - ab_test_start).days, 1)
```

```
# ======================================================================
# CALCULATE MONTHLY INDICATORS
# ======================================================================
def get_monthly_mrr(device, df_chk):
    vol = df_chk.loc['/billing', '1. Traffic (Volume)']
    orders = df_chk.loc['/billing', '2. Purchased Orders']
    cr_new = df_chk.loc['/billing-2', '4. Purchase Rate'] / 100

    aov_old = df_old_billing.loc[device, 'aov_usd']
    aov_new = df_new_billing.loc[device, 'aov_usd']

    actual_mrr = (orders * aov_old / test_duration_days) * 30
    projected_mrr = ((vol * cr_new) * aov_new / test_duration_days) * 30

    return actual_mrr, projected_mrr - actual_mrr

dk_act, dk_uplift = get_monthly_mrr('desktop', df_desktop_chk)
mb_act, mb_uplift = get_monthly_mrr('mobile', df_mobile_chk)
```

```
# ======================================================================
# RESULT
# ======================================================================

df_waterfall = pd.DataFrame({
    'Name': [
        "Revenue from /billing",
        "Growth from desktop",
        "Growth from mobile",
        "Revenue forecast"
    ],
```

```
    'Plotly_Measure': ["absolute", "relative", "relative", "total"], # Column classification
    'Value': [dk_act + mb_act, dk_uplift, mb_uplift, 0]
})

# Auxiliary colum
total_mrr = dk_act + mb_act + dk_uplift + mb_uplift
df_waterfall['Display label'] = [
    f"${dk_act + mb_act:,.0f}",
    f"+${dk_uplift:,.0f}",
    f"+${mb_uplift:,.0f}",
    f"${total_mrr:,.0f}"
]

print("="*85)
print("REVENUE FORECAST RESULT")
print("="*85)
# Export report
display(df_waterfall[['Name', 'Value', 'Display label']])
```

```
# ======================================================================
# VISUALIZE
# ======================================================================
fig = go.Figure(go.Waterfall(
    x = df_waterfall['Name'],
    y = df_waterfall['Value'],
    measure = df_waterfall['Plotly_Measure'],
    text = df_waterfall['Display label'],
    textposition = "outside",
    increasing_marker_color = "#75D187", # Green for the increase
    totals_marker_color = "#4A90E2"      # Blue for the landmarks
))

fig.update_layout(
    title_text = "<b>Monthly Profit Margin (MRR)</b><br><sup>The monthly revenue increases if the /billing-2 page is applied to all t
    yaxis_title = "Revenue (USD / Month)",
    plot_bgcolor = "rgba(240, 246, 255, 1)",
    height = 550
)

fig.show()
```

## V. Conclusion

## 5.1. Hypothesis Testing Results

- **Branch 1 Hypothesis (Navigation Experience & Landing Pages):** There is a severe friction point in the navigation flow, particularly on mobile device. Customers arrive at the landing pages but either lack the incentive or encounter usability hurdles that prevent them from proceeding to the product detail page.
- **Branch 4 Hypothesis (A/B Testing & Projections):** The /billing-2 version vastly outperforms the /billing in terms of the checkout experience across all devices and demonstrates stronger cross-selling capabilities. If /billing-2 is rolled out to 100% of traffic, it is projected that the company will recoup the traffic allocated during the A/B testing phase, generating an additional 1,916 USD in monthly revenue and elevating the total expected monthly revenue to 4,557 USD.

## 5.2. Report Limitations & Recommendations

5.2.1. Limitations

- Access Constraints: Unable to analyze abandoned carts to determine if excessively high cart values or steep shipping fees are the primary causes of customer hesitation and drop-offs.
- Missing Schema Requirements: Currently, orders table only has the `price_usd` column.

→ An order must be successfully paid and completed before the system captures and records its monetary value.

5.2.2. Recommendations for Improvement

- Data Restructuring & Tracking: It is necessary to grant broader data access permissions or re-engineer the data tracking pipeline to capture the cart value of users who abandon their sessions before completing payment.
- Deeper Behavioral Analysis: Conduct a more granular evaluation of the two existing checkout pages once sufficient behavioral data regarding the drop-off segment is collected.

→ Ultimate Goal: Comprehensively optimize the buyer's user experience (UX) at the critical payment decision stage.

## 5.3. Action Plan (By Department)

A. Product & Tech

- (Immediate Priority): Deprecate the original /billing page and redirect 100% of traffic to /billing-2
- (Short-term): Conduct a UI/UX audit of the mobile navigation flow; investigate the root causes behind the underperforming mobile CTAs (Call-to-Actions).
- (Medium-term): Deep-dive into the performance of Lander 3 to establish an optimized standard/template for all other landing pages.

B. Marketing & Growth

- (Immediate Priority): Review the budget allocation for mobile pages with a View Rate of < 20%. Pause ad spend on these pages until the user experience issues are resolved.
- (Short-term): Shift and concentrate the budget toward Lander 3 on Mobile, as it currently leads in conversion rate.
- (Medium-term): Evaluate overall traffic quality to ensure strong "message match" (preventing disconnects between ad creatives and landing page content).

C. Data & Analytics

- (Monitoring): Set up a real-time dashboard tracking Revenue and Conversion Rate (CR) for the /billing-2 page over the next month to verify if it meets the projected targets.
- (Deep Analysis): Implement heatmap tracking across mobile pages to provide the UI/UX team with actionable insights for optimization.